

Stochastic Choice of Basis Functions in Adaptive Function Approximation and the Functional-Link Net

Summary of Igel and Pao, IEEE Transactions on Neural Networks, 1995

1 Introduction

This paper presents a theoretical justification for the random vector version of the functional-link (RVFL) net based on a general approach to adaptive function approximation. The method uses a limit-integral representation of the function to be approximated and evaluates the integral through the Monte-Carlo approach. The paper provides mathematical proof that RVFL networks are universal approximators for continuous functions and that they achieve approximation error convergence at a rate of $O(C/\sqrt{n})$, where n is the number of basis functions, with C independent of n .

2 Methodology

2.1 General Approach to Function Approximation

The key insight of the approach is to represent a continuous function $f \in C(I^d)$ defined on the standard hypercube $I^d = [0, 1]^d \subset \mathbb{R}^d$ as a limit-integral:

$$f(x) = \lim_{\lambda \rightarrow a} \int_V G_{\lambda,x}(X) T[f(X)] dX \quad (1)$$

where:

- λ and X are parameters (with λ being low-dimensional and X multidimensional)
- G is an activation function
- T is an operator defined on $C(I^d)$
- a is a finite or infinite real number
- V is the domain of parameter X

The approximation is achieved through two stages:

1. Approximating the limiting value of the integral by the integral with finite but large λ
2. Obtaining an estimate of the multiple integral using the Monte-Carlo method:

$$\int_V G_{\lambda,x}(X) T[f(X)] dX \approx \frac{|V|}{n} \sum_{k=1}^n G_{\lambda,x}(X_k) T[f(X_k)] \quad (2)$$

where $X_1, \dots, X_n$ is a sample of size n drawn from the uniform distribution on V .

2.2 Random Vector Functional-Link (RVFL) Net

The RVFL net is defined as a single hidden layer feedforward neural network:

$$f_{w_n}(x) = \sum_{k=1}^n a_k g(w_k \cdot x + b_k) \quad (3)$$

where:

- $w_n = (n, a_1, \dots, a_n, b_1, \dots, b_n, w_1, \dots, w_n)$ is the overall parameter of the net
- $w_k \cdot x$ is the inner product of vectors w_k and x
- g is an activation function
- a_k are trainable output layer weights
- $w_k \in \mathbb{R}^d$ and $b_k \in \mathbb{R}$ are randomly assigned and fixed hidden layer parameters

The key distinction between RVFL and general nonlinear perceptron (GNP) is:

- In RVFL: Parameters w_k, b_k of the hidden layer are selected randomly and independently in advance, while parameters a_k of the output layer are learned using simple quadratic optimization
- In GNP: All parameters a_k, w_k, b_k need to be learned using complex nonquadratic optimization

2.3 Algorithm Details

The RVFL learning algorithm proceeds as follows:

Algorithm 1 RVFL Learning Algorithm

- 1: Generate random parameters $w_k \in \mathbb{R}^d$ and $b_k \in \mathbb{R}$ for $k = 1, \dots, n$
 - 2: For w_k : Generate independently and uniformly from $[0, \Omega] \times [-\Omega, \Omega]^{d-1}$
 - 3: For b_k : Set $b_k = -w_k \cdot y_k - u_k$ where y_k is uniformly distributed in I^d and u_k is uniformly distributed in $[-2\Omega, 2\Omega]$
 - 4: Compute the output of hidden nodes for all training samples
 - 5: Find the optimal values for output weights a_k using linear least squares optimization
-

The network training is thus reduced to linear optimization (typically using conjugate gradient method), making it simple and fast.

3 Theoretical Findings

The paper presents three main theoretical results:

3.1 Universal Approximation Property

Theorem 1: For any compact set $K \subset I^d$, $K \neq I^d$ and any absolutely integrable activation function g such that $\int_{\mathbb{R}} g^2(x)dx < \infty$, there exists a sequence of RVFL networks $\{f_{w_n}\}$ and a sequence of probability measures $\{p_{n,\Omega,\alpha}\}$ such that:

$$E \int_K |f(x) - f_{w_n}(x)|^2 dx \rightarrow 0 \text{ as } n \rightarrow \infty \quad (4)$$

Corollary 1: The universal approximation property also holds for any differentiable activation function g with $\int_{\mathbb{R}} [g'(x)]^2 dx < \infty$, which includes commonly used sigmoid functions.

Corollary 2: An alternative form of the RVFL network as a product of univariate functions:

$$f_{w_n}(x) = \sum_{k=1}^n a_k \prod_{i=1}^d g(w_{ki}x_i + b_{ki}) \quad (5)$$

also possesses the universal approximation property.

3.2 Radial Basis Function Networks with Random Parameters

Theorem 2: Neural networks with radial basis functions and random parameters in the form:

$$f_{w_n}(x) = \sum_{k=1}^n a_k g \left(\sum_{i=1}^d w_{ki} (x_i - y_{ki})^2 \right) \quad (6)$$

are also universal approximators for continuous functions on any compact subset of I^d .

3.3 Approximation Error Rate

Theorem 3: For any Lipschitz continuous function $f \in C(I^d)$ satisfying $|f(x) - f(y)| \leq K\|x - y\|$ for some constant $K > 0$, any compact set $K \subset I^d$, $K \neq I^d$, and any activation function g_β with appropriate support, there exists a sequence of RVFL networks $\{f_{w_n}\}$, a sequence of probability measures $\{p_{n,\Omega,\alpha}\}$, and a constant $C_{f,g,\Omega,\alpha,\beta,d}$ such that:

$$E \int_K |f(x) - f_{w_n}(x)|^2 dx \leq \frac{C_{f,g,\Omega,\alpha,\beta,d}}{n} \quad (7)$$

This means the approximation error converges to zero at a rate of $O(C/\sqrt{n})$ with C independent of n . This is significantly better than the approximation by linear combinations of fixed basis functions, which gives an error on the order of $O(1/n^{1/d})$.

4 Proof Sketch

The proofs rely on several key steps:

1. Representing a function to be approximated as the limiting value of a multidimensional integral using window transformations

2. Using a kernel that concentrates a function at specific points or surfaces of its domain
3. Replacing the kernel by the activation function g
4. Using Monte-Carlo methods to approximate the integral
5. Establishing bounds on the approximation error

The mathematical foundation relies on representing the function using a window transformation $h_x(y, w)$ that approaches the multidimensional delta function as parameters approach infinity. This leads to a limit-integral representation which is then evaluated using Monte-Carlo integration.

5 Numerical Examples and Applications

The paper describes several practical applications where RVFL networks demonstrated superior performance compared to traditional neural networks:

5.1 Optical Ellipsometry Problem

In molecular beam epitaxy film thickness monitoring, RVFL networks successfully solved the complex inversion problem of determining film refractive index and thickness from ellipsometry measurements:

- Input variables: Two pairs of ψ and Δ angular measurements
- Output variables: n and κ (real and imaginary parts of refractive index) and two film thicknesses d_0 and d_1
- Training: 10^4 training patterns (sparse set in 4D space)
- Results: Refractive index n evaluated with error less than 0.1%
- Performance: Training achieved in about six hours on a SPARC 2 workstation, whereas backpropagation failed to achieve satisfactory results
- Network size: 50-200 basis functions

5.2 Comparison with Backpropagation

A comparison table shows RVFL outperforming backpropagation (BP) across various tasks:

Task	Dim.	Size	RVFL Time	BP Time	Accuracy
Pattern Rec.	25-5	100	46 sec	31.2 min	Equal
Image Gen.	2-32	400	165 sec	240.8 min	Equal
Ellipsometry	4-4	9000	6 hrs	*	RVFL better

* Backpropagation failed to achieve comparable accuracy

6 Implications and Future Directions

The theoretical and practical results presented have several important implications:

6.1 Computational Efficiency

RVFL networks offer significant advantages over traditional neural networks:

- Linear learning (only output weights are trained) is simple and fast
- Approximation error converges at a rate of $O(C/\sqrt{n})$, which is the same as GNP but with much simpler training
- RVFL avoids the exponential growth in the number of basis functions that occurs with fixed basis functions

6.2 Enhancements for Improved Accuracy

The paper suggests methods to enhance the accuracy of RVFL networks:

- Using ensemble methods with multiple RVFL networks
- Applying the Pincus formula for estimating the optimal parameters:

$$\theta_j \approx \frac{\sum_{k=1}^K \theta_{kj} \exp[-\lambda E_{n,N}(\theta_k)]}{\sum_{k=1}^K \exp[-\lambda E_{n,N}(\theta_k)]}, j = 1, \dots, 2n \quad (8)$$

where $\theta_k = (\theta_{k1}, \dots, \theta_{k,2n})$ are randomly generated parameter sets

- Using variance reduction methods in Monte-Carlo integration where specific information about the function is available

6.3 Broader Impact

The stochastic approach based on limit-integral representation with Monte-Carlo evaluation opens new possibilities for efficient approximation of multivariate functions:

- The approach can be extended to other types of neural networks
- It provides a mathematical foundation for understanding why randomized networks can be effective
- It suggests that combining multiple independent RVFL estimates can be more effective than increasing the number of basis functions in a single network

The theoretical results confirm that RVFL networks represent a practical compromise between fully trainable networks and networks with fixed basis functions, combining both simplicity of learning and efficiency of representation.